Ejercicios de ILP
Paralelismo a nivel de instrucción

IIC2343 — Arquitectura de Computadores

II/2026

Supuestos para todos los ejercicios

Computador básico con *pipeline*. No hay instrucciones previas en el *pipeline*.

Etapas y adelantamiento

- Etapas: IF, ID, EX, MEM, WB.
- Hay *forwarding* desde `EX/MEM` y desde `MEM/WB` hacia los operandos de la ALU en EX.
- Una escritura a memoria recibe el dato por *forwarding* desde `MEM/WB` hacia MEM.
- El banco de registros se escribe al inicio de WB. Quien esté en ID en ese mismo ciclo lee el valor nuevo. Quien lo necesite en EX en ese ciclo usa *forwarding*.

Stalling y saltos

- Si la instrucción siguiente usa en la ALU, o en la comparación de un salto, un registro recién leído de memoria, el *forwarding* llega tarde. Hay que hacer *stalling* con un `NOP`. El dato queda disponible al **terminar** MEM.
- Una escritura a memoria no agrega un ciclo de *stalling*.
- El predictor asume que **ningún** salto se toma, incluido `JMP`.
- `JEQ A, B, Dir` compara en EX. L_{PC} se activa en MEM. Si el salto se toma, se hace *flush* de las tres instrucciones ya ingresadas.
- El *stalling* es por *software*: el `NOP` se dibuja en el diagrama.

Ejercicios teóricos

T1. Clasifique el *hazard*

Para cada fragmento indique si hay *hazard* de datos, de control o estructural, o si no hay ninguno. Si lo hay, nombre el registro o recurso en conflicto y el mecanismo que lo resuelve: *forwarding*, *stall* o *flush*.

(a)

```
ADD A, 1
MOV B, A
```

(b)

```
MOV A, (var)
SUB A, 1
```

(c)

```
JMP fin
ADD A, 1
MOV B, A
SUB B, 1
```

(d) En un mismo ciclo, una instrucción está en MEM leyendo la memoria de datos y otra está en IF leyendo la memoria de instrucciones. ¿Hay un *hazard* estructural? Justifique con la organización de las memorias.

Solución T1

1. **Datos, registro `A`.** `ADD` escribe `A` y `MOV` lo lee. Se resuelve con *forwarding* desde `EX/MEM` hacia EX: cuando `MOV` está en EX, `ADD` está en MEM. No hay *stall*.
2. **Hazard de datos sobre `A`.** El dato se lee al **final** de MEM, pero `SUB` lo necesita al **inicio** de EX, en el mismo ciclo. El *forwarding* llega tarde. Hay que insertar un `NOP` (*stalling* por software) y luego adelantar desde `MEM/WB`.
3. **Control.** `JMP` siempre se toma y el predictor asume que no. Cuando `JMP` está en MEM se hace *flush* de las tres instrucciones que ya entraron: `ADD` en EX, `MOV` en ID y `SUB` en IF.
4. **No hay *hazard* estructural.** La memoria de instrucciones y la de datos están separadas, así que IF y MEM pueden leer en el mismo ciclo.

T2. *Stalling*

Considere el fragmento

```
MOV B, (Dir)
ADD A, B
```

1. ¿Por qué el *forwarding* desde `EX/MEM` no alcanza a entregarle `B` a la ALU de `ADD`?
2. Tras insertar el `NOP`, indique en qué etapa está cada instrucción en el ciclo en que `B` ya está disponible para la ALU.
3. Si el *stalling* es por *hardware*, ¿desde qué etapa puede detectarse? Escriba una combinación de señales que lo detecte. Puede usar `LoadB`, `AluB`, `RegIn`, `DOUT` y el registro `ID/EX`.

Solución T2

1. Cuando `ADD` está en EX, `MOV` está en MEM. `B` se está leyendo en ese ciclo y todavía no está guardado en `EX/MEM`: ese registro se actualiza al **terminar** MEM. La ALU necesita el operando al inicio de EX, así que el adelantamiento desde `EX/MEM` llega tarde.
2. Con el `NOP` en medio, en el ciclo en que `B` ya sirve para la ALU: `MOV` está en **WB** y `ADD` está en **EX**. El dato se adelanta desde `MEM/WB`.
3. Se detecta en **ID**, mirando la instrucción anterior en `ID/EX`:

```
ID.AluB == B && ID/EX.LoadB == 1 && ID/EX.RegIn == DOUT
```

T3. El `JMP` también pierde tres ciclos

`JMP fin` siempre se toma. Aun así, con los supuestos de este computador, la penalización es de tres ciclos.

1. Cuando el `JMP` está en MEM, ¿en qué etapa está cada una de las tres instrucciones que se flushean?
2. La dirección de destino ya se conoce al decodificar la instrucción. Explique por qué eso no evita la penalización de tres ciclos en esta implementación.
3. ¿En qué etapa habría que escribir `PC` para que un `JMP` perdiera un solo ciclo? ¿Por qué la penalización no puede ser cero?

Solución T3

1. Con el `JMP` en MEM, las tres instrucciones ya ingresadas están en **EX**, **ID** e **IF**. Esas tres se flushean. El destino se lee recién en el ciclo siguiente.
2. En esta implementación L_{PC} solo se activa en MEM. Aunque la dirección se conoce en ID, no hay un camino que escriba `PC` en esa etapa. Mientras el `JMP` recorre ID, EX y MEM, IF sigue trayendo las instrucciones siguientes.
3. Habría que escribir `PC` al **final de ID**. La penalización queda en un ciclo: la instrucción siguiente alcanza a fetchearse en paralelo con el ID del `JMP`. No puede ser cero, porque durante IF todavía no se sabe que la instrucción es un salto.

T4. Escrituras y banco de registros

(a) En el fragmento siguiente, ¿cuántos ciclos extra agrega la dependencia? Indique desde qué registro de *pipeline* y hacia qué etapa se adelanta `A`.

```
ADD A, 3
MOV (Dir), A
```

(b) En el fragmento siguiente, `SUB` usa el `A` producido por `MOV A, 3`. ¿Hace falta *forwarding*? Justifique indicando qué etapa de `MOV A, 3` coincide con el ID de `SUB`.

```
MOV A, 3
MOV B, 1
ADD B, 1
SUB A, 1
```

Solución T4

1. **Cero ciclos extra.** La escritura necesita `A` en MEM, no en EX. Cuando `MOV (Dir), A` está en MEM, `ADD` está en WB. `A` se adelanta desde `MEM/WB` hacia la entrada de datos de la memoria.
2. **No hace falta *forwarding*.** Entre `MOV A, 3` y `SUB` hay dos instrucciones. Cuando `SUB` está en ID, `MOV A, 3` está en WB. El banco se escribe al inicio de ese ciclo y ID alcanza a leer el valor nuevo.

T5. Estimación en función de N

Un programa tiene N instrucciones. En el computador de estos ejercicios:

- el 15 % de las instrucciones son saltos y, de esos saltos, el 40 % se toma;
- el 25 % son lecturas de memoria y, de esas lecturas, el 60 % usa el dato en la ALU de la instrucción siguiente;
- el 20 % son escrituras de memoria y, de esas escrituras, el 50 % escribe un dato calculado por la instrucción anterior.

Estime, en términos de N, cuántos ciclos tarda el programa. Parta del caso sin penalizaciones y sume por separado los ciclos extra de cada efecto. Las escrituras solo aportan si, con estos supuestos, agregan ciclos.

Solución T5

Sin penalizaciones el programa termina en $N + 4$ ciclos. A eso se suman solo los efectos que agregan ciclos:

- Saltos tomados: $3 \times 0{,}15 \times 0{,}40\, N = 0{,}18N$ ciclos de *flush*.
- Lectura de memoria usada en la instrucción siguiente: $1 \times 0{,}25 \times 0{,}60\, N = 0{,}15N$ ciclos de *stalling*.
- Escrituras: el dato se adelanta. **No agregan ciclos.**

$$N + 4 + 0{,}18N + 0{,}15N = 1{,}33N + 4$$

T6. Resolver el salto en EX

Se modifica la unidad de *hazards* de control: L_{PC} pasa a activarse en EX, tanto para `JEQ` como para `JMP`. La comparación de `JEQ A, B, Dir` sigue haciéndose en EX.

1. ¿Cuántos ciclos se pierden ahora si la predicción falla? Indique qué instrucciones se flushean, según la etapa en la que están.
2. Si, además, un `JMP` pudiera escribir `PC` al **final de ID**, ¿de cuántos ciclos sería su penalización? Explique por qué sigue siendo imposible que sea cero.
3. Señale un efecto de mover la decisión a EX sobre el periodo del reloj.

Solución T6

1. Se pierden **dos** ciclos. Con el salto en EX, las instrucciones que ya entraron están en ID y en IF. Esas dos se flushean.
2. La penalización de un `JMP` queda en **un** ciclo. Durante su ID, IF ya alcanzó a traer la instrucción siguiente; al final de ID se redirige `PC` y solo esa se descarta. No puede ser cero: en IF la instrucción todavía no está decodificada, así que no se sabe que es un salto.
3. EX hace ahora la comparación y además decide L_{PC}. Si esa lógica alarga la etapa, **sube el periodo** del reloj.

T7. `PUSH` y `POP` de vuelta en el *pipeline*

Se reincorporan `PUSH A` y `POP A`. `POP` lee la memoria en MEM. `SP` se actualiza en WB, con la misma convención del banco de registros.

1. En `POP A` seguido de `ADD A, 1`, ¿cuál es el registro en conflicto? ¿Basta el *forwarding* o hace falta un *stall*? Compare las etapas de ambas instrucciones en el ciclo en que el dato queda disponible.
2. Escriba una secuencia de dos instrucciones cuyo conflicto sea `SP`, y no el dato que se lee o se escribe. Indique en qué etapas se cruzan.
3. `PUSH A` seguido de `MOV (Dir), B` escribe dos veces la memoria de datos. ¿Es un *hazard* estructural? Justifique con cuántas escrituras a memoria de datos ocurren en un mismo ciclo.

Solución T7

1. El conflicto es `A`. `POP` deja el dato al final de MEM y `ADD` lo necesita al inicio de EX: el *forwarding* llega tarde. **Hace falta *stalling***. Tras el `NOP`, `POP` está en WB y `ADD` en EX, y `A` se adelanta desde `MEM/WB`.
2. Ejemplo: `PUSH A` y después `PUSH B`. El conflicto es `SP`, no el dato. El primer `PUSH` escribe `SP` en WB. Sin *stall*, el segundo está en EX cuando el primero está en MEM, y el `SP` nuevo todavía no existe: no hay un resultado en `EX/MEM` que adelantar. Tras un `NOP`, el segundo está en EX cuando el primero está en WB y ahí sí se adelanta.
3. **No es estructural.** En un ciclo solo una instrucción está en MEM, así que hay una sola escritura a la memoria de datos. Las dos escrituras ocurren en ciclos seguidos.

T8. Bundles de un procesador 2-*issue*

El compilador emite *bundles* de dos instrucciones, al estilo VLIW. El *forwarding* solo lee registros de *pipeline* escritos el ciclo anterior. Hay una sola ALU y la memoria de datos tiene un solo puerto. **(a)** Nombre tres modificaciones mínimas de *hardware* para que el *fetch* y el *decode* acepten un *bundle* de dos instrucciones en el mismo ciclo. **(b)** Diga si cada *bundle* puede emitirse junto. Si no puede, indique si el conflicto es estructural o de datos.

1

```
ADD A, B
SUB B, 1
```

2

```
MOV A, (x)
MOV (y), B
```

3

```
MOV A, (x)
ADD A, 1
```

Solución T8

(a) Tres cambios para que entren dos instrucciones en el mismo ciclo:

- memoria de instrucciones de doble ancho, o con dos puertos de lectura;
- dos decodificadores, y un `IF/ID` que transporte las dos instrucciones;
- banco de registros con cuatro puertos de lectura, porque ambas leen sus operandos en ID.

(b)

1. **No. Estructural:** las dos usan la única ALU en el mismo ciclo.
2. **No. Estructural:** una lee y la otra escribe la memoria de datos, y hay un solo puerto.
3. **No. Datos:** `MOV` lee `A` de memoria y `ADD` lo necesita en el mismo ciclo. El *forwarding* llega tarde: solo ve registros escritos el ciclo anterior.

Ejercicios de diagrama

Diagramas de *pipeline*

En D1, D2 y D3 dibuje una fila por instrucción, en orden de ejecución. Inserte el `NOP` como fila propia donde corresponda.

- En cada celda anote la etapa: IF, ID, EX, MEM o WB.
- *Forwarding*: flecha desde la etapa que produce el dato hacia la etapa que lo consume.
- *Flush*: tache las etapas que no llegan a ejecutarse.
- El total de ciclos es el ciclo en que la última instrucción termina WB.
- Puede sobrar alguna fila o alguna columna.

D1. Adelantamiento y un *stall*

`x` e `y` son direcciones distintas. No hay saltos: el contenido de memoria no cambia el diagrama. Marque cada *forwarding* y el `NOP`, e indique el total de ciclos.

```
MOV A, 4
ADD A, 1
MOV B, (x)
ADD A, B
MOV (y), A
```

Ciclos totales: _____

Instr.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Solución D1

10 ciclos. El `NOP` es el *stalling*: `B` se lee de memoria y la instrucción siguiente lo usa.

Instr.	1	2	3	4	5	6	7	8	9	10
MOV A, 4	IF	ID	EX	MEM	WB					
ADD A, 1		IF	ID	EX	MEM	WB				
MOV B, (x)			IF	ID	EX	MEM	WB			
NOP				IF	ID	EX	MEM	WB		
ADD A, B					IF	ID	EX	MEM	WB	
MOV (y), A						IF	ID	EX	MEM	WB

- Ciclo 4: `EX/MEM` de `MOV A, 4` → EX de `ADD A, 1`.
- Ciclo 7: `MEM/WB` del *load* → EX de `ADD A, B`. `A` ya está en el banco: se escribió en el WB del ciclo 6.
- Ciclo 9: `MEM/WB` de `ADD A, B` → MEM de `MOV (y), A`.

D2. Un salto que hay que evaluar

Determine si `JEQ` se toma. Incluya en el diagrama las instrucciones que se flushean y marque el *forwarding* hacia la comparación. Indique el total de ciclos.

```
MOV A, 2
MOV B, 2
JEQ A, B, fin
MOV A, 7
ADD A, 1
MOV B, 3
fin:
MOV A, 0
MOV B, 0
```

Instr.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Ciclos totales: _____

Solución D2

`A = 2` y `B = 2`, así que `JEQ` **se toma**. **12 ciclos.**En naranja, las etapas que se flushean en el ciclo 6.

Instr.	1	2	3	4	5	6	7	8	9	10	11	12
MOV A, 2	IF	ID	EX	MEM	WB							
MOV B, 2		IF	ID	EX	MEM	WB						
JEQ A, B, fin			IF	ID	EX	MEM	WB					
MOV A, 7				IF	ID	EX						
ADD A, 1					IF	ID						
MOV B, 3						IF						
MOV A, 0							IF	ID	EX	MEM	WB	
MOV B, 0								IF	ID	EX	MEM	WB

- Ciclo 5, en el EX de `JEQ`: `B` llega desde `EX/MEM` y `A` desde `MEM/WB`.
- Ciclo 6: `JEQ` está en MEM, el salto se toma y se flushean las tres instrucciones siguientes.
- `fin` entra a IF en el ciclo 7. `MOV B, 0` termina WB en el ciclo 12.

D3. Lectura, escritura y dos saltos

`x` parte en 4. Evalúe `JEQ` y `JMP` por separado. El diagrama debe mostrar el *stall*, los *forwarding* (puede haber más de uno en el mismo ciclo) y el *flush*. La grilla está en la diapositiva siguiente.

```
DATA:
  x  4
CODE:
  MOV B, 0
  MOV A, (x)
  ADD A, 1
  MOV (x), A
  JEQ A, B, fin
  JMP fin
  MOV A, 9
  MOV B, 9
  ADD A, B
fin:
  MOV A, 0
```

Qué tiene que verse

- El `NOP` del *stalling*, en su posición.
- Flechas de *forwarding* hacia EX y, en la escritura, hacia MEM.
- El salto que no se toma, sin *flush*.
- El salto que sí se toma, con las tres instrucciones tachadas.
- El ciclo en que `MOV A, 0` termina WB.

D3. Grilla

Instr.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

Ciclos totales: _____

Solución D3

`x = 4`, entonces `A` queda en 5 y `B` en 0: `JEQ` **no** se toma. `JMP` **sí**. **15 ciclos.**

Instr.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
MOV B, 0	IF	ID	EX	MEM	WB										
MOV A, (x)		IF	ID	EX	MEM	WB									
NOP			IF	ID	EX	MEM	WB								
ADD A, 1				IF	ID	EX	MEM	WB							
MOV (x), A					IF	ID	EX	MEM	WB						
JEQ A, B, fin						IF	ID	EX	MEM	WB					
JMP fin							IF	ID	EX	MEM	WB				
MOV A, 9								IF	ID	EX					
MOV B, 9									IF	ID					
ADD A, B										IF					
MOV A, 0											IF	ID	EX	MEM	WB

- Ciclo 6: `MEM/WB` del *load* $\rightarrow$ EX de `ADD A, 1`.
- Ciclo 8, dos flechas desde el mismo WB: hacia MEM de `MOV (x), A` y hacia EX de `JEQ`. `B` ya está en el banco.
- Ciclo 9: `JEQ` en MEM, $5 \neq 0$, no hay *flush*.
- Ciclo 10: `JMP` en MEM. Se flushean `MOV A, 9` (EX), `MOV B, 9` (ID) y `ADD A, B` (IF).